

Politechnika Warszawska

W Y D Z I A Ł E L E K T R Y C Z N Y



Praca dyplomowa magisterska

na kierunku Elektrotechnika
w specjalności Systemy Wbudowane

Synteza trajektorii fazowych oscylatora chaotycznego metodami generatywnymi

inż. Lidia Kocon
numer albumu 318921

promotor
dr. inż. Łukasz Makowski

WARSZAWA 2026

**Synteza trajektorii fazowych oscylatora chaotycznego metodami
generatywnymi
Streszczenie**

Słowa kluczowe:

**Synthesis of Phase-Space Trajectories of a Chaotic Oscillator Using Generative
Methods
Abstract**

Keywords:

Spis treści

1	Wstęp	9
2	Teoria chaosu	11
2.1	Entropia Shannona	11
2.1.1	Zasada działania	12
2.2	Korelacja Pearsona	13
2.2.1	Zasada działania	13
2.3	Wykładnik Lapunowa	14
2.3.1	Zasada działania	14
2.3.2	Algorytm Rosensteina	15
2.4	Atraktor Lorenza	16
2.4.1	Zasada działania	16
2.5	Sieci GAN	17
2.5.1	Zasada działania	18
2.5.2	Generator i dyskryminator	19
2.6	Sieci z pamięcią LSTM	19
2.6.1	Bramki informacyjne	19
2.6.2	Jednowymiarowe sieci splotowe Conv1D	20
3	Generowanie chaosu	21
3.1	Dane wejściowe	21
3.2	Kod źródłowy	22
3.2.1	Importowanie bibliotek	22
3.2.2	Inicjalizacja klasy głównej	22
3.2.3	Przygotowanie Danych	23
3.2.4	Struktura Generatora	24
3.2.5	Struktura Dyskryminatora	25
3.2.6	Procedura Treningowa	26
3.2.7	Pętla Ucząca	27
3.2.8	Generowanie przebiegów	28
3.3	Uzasadnienie wyboru architektury GAN	28

4 Dowód chaosu	31
4.1 Analiza Chaosu	31
4.2 Wykładnik Lapunowa	32
4.2.1 Analiza wyników	33
4.3 Analiza entropii	33
4.3.1 Analiza wyników	34
4.4 Badanie korelacji	35
4.4.1 Analiza wyników	35
4.5 Atraktor Lorenza	36
4.5.1 Analiza wyników	37
5 Podsumowanie	39
Bibliografia	41
Spis rysunków	45

Rozdział 1

Wstęp

Rozdział 2

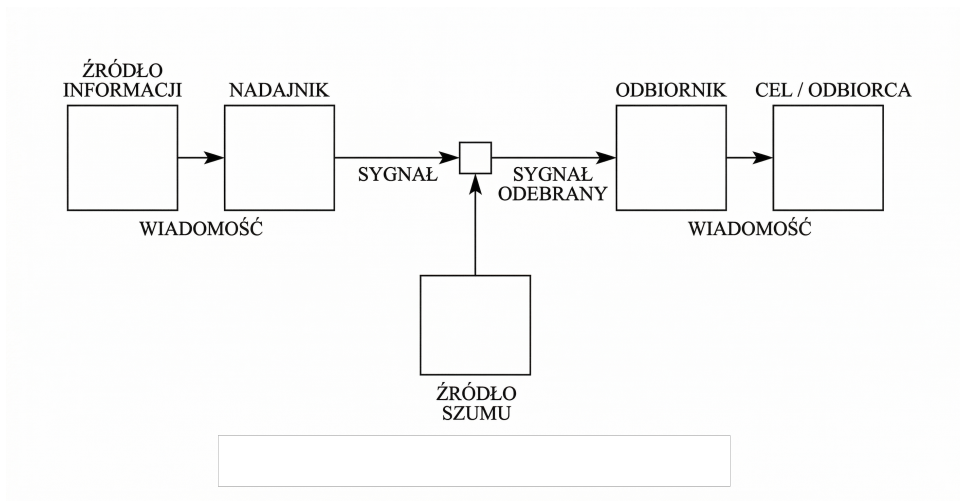
Teoria chaosu

Na początku był Chaos. Któż zdoła powiedzieć dokładnie, co to był Chaos? Niejedni widzieli w nim jakąś istotę boską, ale bez określonego kształtu. Inni – a takich było najwięcej – mówili, że to wielka otchłań, pełna siły twórczej i boskich nasieni, jakby jedna masa nieuporządkowana, ciężka i ciemna, mieszanina ziemi, wody, ognia i powietrza [30].

Pojęcie chaosu, choć pierwotnie kojarzone z boską istotą pozbawioną formy lub bezkresną otchłanią, we współczesnej nauce zyskało nową i bardziej precyzyjną definicję. Obecnie zjawisko to bada się w różnych dziedzinach, takich jak biologia, gdzie opisuje się za jego pomocą złożone ekosystemy, czy informatyka, w której odnosi się ono do systemów wysoce nieprzewidywalnych i trudnych do sterowania. Największe znaczenie termin ten posiada jednak w fizyce oraz matematyce, gdzie wiąże się z nieliniowymi układami dynamicznymi. Układy te charakteryzują się ogromną wrażliwością na warunki początkowe, co oznacza, że nawet najmniejsze zmiany parametrów startowych prowadzą do uzyskania innych wyników końcowych. Zjawisko to jest powszechnie znane pod nazwą efektu motyla [39]. Ze względu na skomplikowaną naturę zjawisk chaotycznych, do ich dokładnego badania i modelowania niezbędne jest wykorzystanie odpowiednich narzędzi analitycznych. Z tego powodu poniżej przedstawiono podstawy teoretyczne, które pozwalają na zrozumienie metod pomiaru nieuporządkowania badanych układów oraz sposobów ich symulowania za pomocą algorytmów uczenia maszynowego.

2.1 Entropia Shannona

W 1948 roku ujrzał światło dzienne artykuł zatytułowany „A Mathematical Theory of Communication” [35] napisany przez amerykańskiego matematyka i inżyniera Claude E. Shannon’a, gdzie zaprezentowano podstawy nowej dziedziny zwaną teorią informacji. Wprowadzono tam po raz pierwszy matematyczny sposób mierzenia ilości informacji, określaną jako entropia informacyjna, powszechnie znaną dziś pod nazwą entropii Shannona.



Rysunek 1. Schemat ogólnego systemu komunikacji

2.1.1 Zasada działania

Entropia jest miarą stopnia nieuporządkowania systemu. Im wyższa jest jej wartość, tym bardziej chaotyczny jest układ. Entropia Shannona określa, jak trudno jest przewidzieć kolejną wartość w badanym zbiorze danych. Wartość ta zależy bezpośrednio od częstości występowania każdego z elementów. Do obliczenia entropii wymagana jest znajomość całego zbioru oraz rozkładu prawdopodobieństwa [4] [41]. Podczas analizy sygnałów, w pierwszej kolejności dzieli się zebrane dane na równe przedziały, najczęściej przy pomocy histogramu. Następnie wyznacza się prawdopodobieństwo $P(x)$, określające szansę trafienia wartości do poszczególnych przedziałów [2] [37].

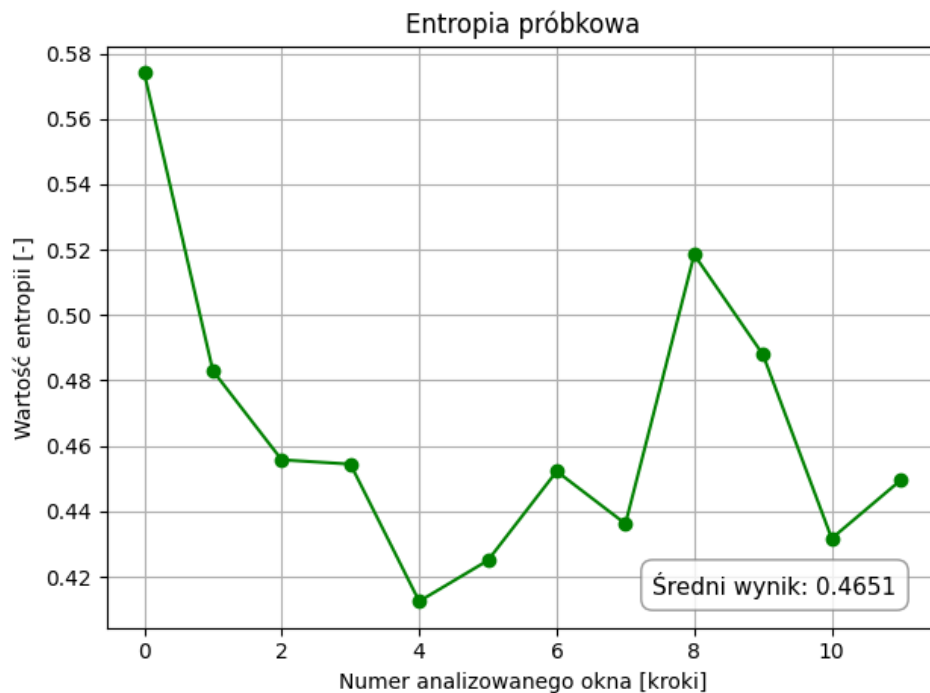
Gdy badany sygnał nie zmienia się i wszystkie odczyty wpadają do tego samego przedziału, kolejny wynik jest znany. W takiej sytuacji nie występuje żadna losowość, co oznacza, że entropia wynosi zero. W przypadku układów o dużej zmienności, odczyty trafiają do wielu różnych przedziałów. Im bardziej równomiernie dane wypełniają przedziały, tym trudniej jest przewidzieć następną wartość, co oznacza, że wartości entropii rośnie. Osiąga ona swoją maksymalną wartość wtedy, gdy rozkład danych jest całkowicie równomierny [32].

Dla zmiennej losowej przyjmującej n różnych wartości, z których każda występuje z pewnym prawdopodobieństwem p_i , entropię H opisuje się następującym równaniem:

$$H = - \sum_{i=1}^n p_i \log_b(p_i)$$

Gdzie:

- p_i to prawdopodobieństwo zajścia i -tego zdarzenia,
- suma wszystkich prawdopodobieństw p_i dla całego układu wynosi zawsze 1
- b to podstawa logarytmu, która bezpośrednio określa jednostkę miary informacji.



Rysunek 2. Entropia Shannona

Sumowanie przeprowadza się wyłącznie dla przedziałów, które nie są puste. Użycie logarytmu o podstawie dwa sprawia, że ostateczny wynik podaje się w bitach. Im większa jest uzyskana liczba bitów, tym bardziej złożony i nieprzewidywalny jest układ [5] [10] [9].

2.2 Korelacja Pearsona

Współczynnik korelacji liniowej Pearsona stanowi jedną z najpopularniejszych miar stosowanych w statystyce do określania poziomu i kierunku zależności liniowej pomiędzy dwiema zmiennymi. Narzędzie to wyznacza się w celu zweryfikowania, czy zmiana jednej wielkości pociąga za sobą proporcjonalną zmianę drugiej.

2.2.1 Zasada działania

Wartość opisywanego wskaźnika mieści się w zamkniętym przedziale od -1 do 1. Dodatni wynik oznacza, że wzrostowi wartości jednej zmiennej odpowiada wzrost wartości drugiej zmiennej. Z kolei ujemna korelacja występuje wtedy, gdy rosnące wartości jednej danej wiążą się ze spadkiem wartości drugiej. Im wartość bezwzględna znajduje się bliżej jedynki, tym silniejsza jest obserwowana relacja liniowa, gdy wynik równy jest zero oznacza to całkowity brak takiego powiązania [42] [29] [8]. Należy jednak uwzględnić fakt, że brak zależności liniowej nie wyklucza całkowicie istnienia związku o charakterze nieliniowym. Prawidłowe zastosowanie

tego narzędzia wymaga wcześniejszego upewnienia się, że w badanej próbie nie występują wartości skrajne, a analizowane parametry charakteryzują się rozkładem normalnym.

Współczynnik korelacji pomiędzy dwiema zmiennymi oblicza się, dzieląc kowariancję, czyli matematyczną miarę odchylenia badanych elementów od idealnego trendu liniowego, przez iloczyn odchyleń standardowych tych zmiennych [3] [21]. Zależność tę określa się za pomocą następującego równania matematycznego:

$$r = \frac{\sum_{i=1}^n (x_i - \bar{x})(y_i - \bar{y})}{\sqrt{\sum_{i=1}^n (x_i - \bar{x})^2} \sqrt{\sum_{i=1}^n (y_i - \bar{y})^2}} \quad (1)$$

Gdzie:

- x_i oraz y_i to poszczególne obserwacje zgromadzone dla badanych zmiennych,
- $\bar{x}$ oraz $\bar{y}$ reprezentują średnie arytmetyczne wyliczone dla badanych zmiennych,
- n określa całkowitą licznosc analizowanej próby danych.

2.3 Wykładnik Lapunowa

Pojęcie wykładnika Lapunowa zostało sformułowane w 1892 roku przez rosyjskiego uczonego Aleksandra Michajłowicza Lapunowa w jego przełomowej rozprawie doktorskiej pod tytułem „The general problem of the stability of motion” [22]. Autor poszukiwał w niej matematycznych metod pozwalających na precyzyjne określenie stabilności układów nieliniowych, co doprowadziło do zdefiniowania tzw. liczb charakterystycznych, znanych dziś powszechnie jako wykładniki Lapunowa [6] [40].

2.3.1 Zasada działania

Wykładnik Lapunowa pozwala ocenić tempo, w jakim dwie początkowo bardzo bliskie sobie trajektorie zbliżają się do siebie lub oddalają w przestrzeni czasu [27] [28] [1]. Dodatni wykładnik Lapunowa wskazuje na obecność chaosu, ponieważ oznacza, że trajektorie oddalają się od siebie w sposób wykładniczy. Zjawisko to opisuje się wzorem:

$$\Delta_k \approx \Delta_0 e^{\lambda_{\max} k} \quad (2)$$

Gdzie:

- $\lambda_{\max}$ to największy wykładnik Lapunowa,
- Δ_0 to odległość początkowa między trajektoriami,
- Δ_k to odległość między trajektoriami po k krokach czasowych.

Zakłada się, że trajektorie nie mogą oddalić się od siebie bardziej, niż wynosi rozmiar całego atraktora. Z tego powodu podany wzór stosuje się tylko dla małych wartości k . Po zlogarytmowaniu obu stron uzyskuje się równanie prostej:

Odległość między dwiema wartościami początkowymi:

$$\begin{array}{c} \varepsilon \\ \hline x_0 \qquad \qquad x_0 + \varepsilon \end{array}$$

Po N iteracjach:

$$\begin{array}{c} \varepsilon e^{N\lambda(x_0)} \\ \hline f^N(x_0) \qquad \qquad f^N(x_0 + \varepsilon) \end{array}$$

Rysunek 3. Wizualizacja dywergencji początkowych trajektorii

$$\ln \Delta_k = \ln \Delta_0 + k\lambda_{\max} \quad (3)$$

W praktyce największy wykładnik Lapunowa to współczynnik kierunkowy prostej dopasowanej do tego równania [26] [7] [44].

2.3.2 Algorytm Rosensteina

Algorytm Rosensteina jest to metoda numeryczna szacowania największego wykładnika Lapunowa ze skończonych szeregów czasowych [16].

Metoda zakłada, że dla wybranego punktu na trajektorii wyszukuje się jego najbliższego sąsiada w przestrzeni i bada się, w jaki sposób odległość między tymi punktami rośnie w kolejnych dyskretnych krokach czasowych k . Rozchodzenie się badanych sąsiadów opisuje funkcja:

$$d_j(k) \approx C_j e^{\lambda_1 k \Delta t}$$

Po zlogarytmowaniu obu stron otrzymuje się zależność liniową:

$$\ln(d_j(k)) \approx \ln(C_j) + \lambda_1 k \Delta t$$

Aby uśrednić zakłócenia w danych, oblicza się średnią po wielu zidentyfikowanych parach sąsiadów M :

$$y(k) = \frac{1}{M} \sum_{j=1}^M \ln(d_j(k)) \approx \lambda_1 k \Delta t + C$$

Największy wykładnik Lapunowa λ_1 określa się ostatecznie jako nachylenie prostej dopasowanej do wykresu średniego tempa oddalania się punktów $y(k)$ w funkcji czasu [16].

2.4 Atraktor Lorentza

Atraktor Lorentza stanowi powszechnie znany przykład układu chaotycznego, którego odkrycia dokonano w 1963 roku podczas badań nad prognozowaniem pogody. Zjawisko to zostało zaobserwowane przez amerykańskiego meteorologa Edwarda Nortona Lorentza w trakcie prowadzonych symulacji komputerowych. Zauważono wówczas, że minimalna zmiana danych wejściowych, polegająca na zaokrągleniu jednej z liczb, doprowadziła do wygenerowania zupełnie innego wyniku końcowego. Na tej podstawie udowodniono, że w niektórych systemach nawet najmniejsze zmiany rosną w tempie lawinowym, co potocznie określa się jako efekt motyla [19] [36].

2.4.1 Zasada działania

Układ składa się z trzech nieliniowych równań różniczkowych. Zostały one pierwotnie wyprowadzone w celu uproszczonego opisu zjawiska konwekcji termicznej, czyli ruchu płynu podgrzewanego od dołu i chłodzonego z góry. Zależności te prezentują się następująco:

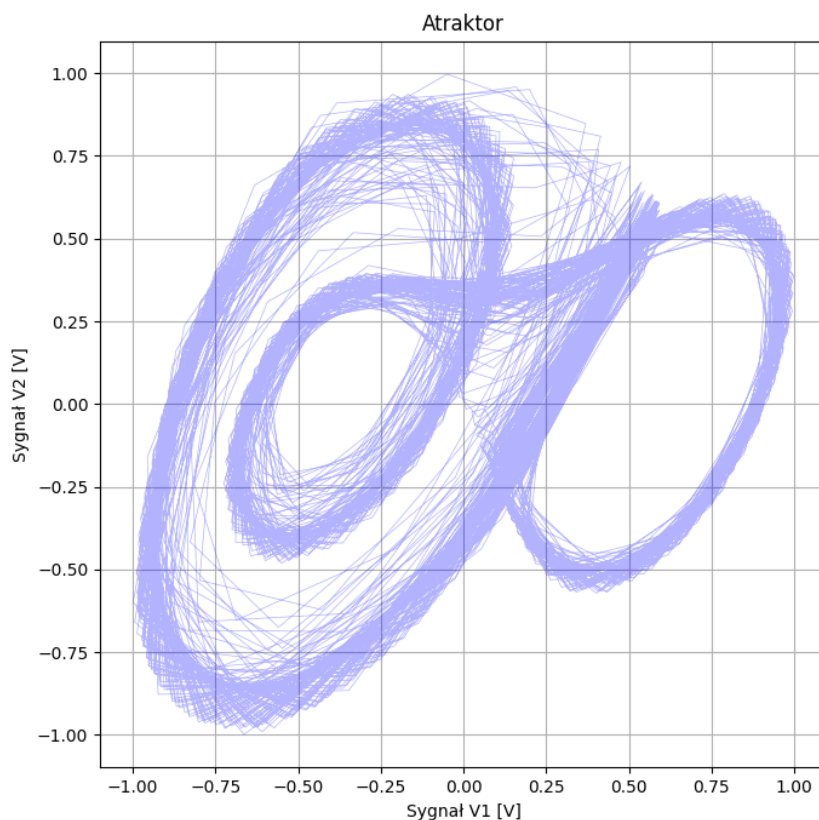
$$\begin{aligned}\frac{dx}{dt} &= \sigma(y - x) \\ \frac{dy}{dt} &= x(\rho - z) - y \\ \frac{dz}{dt} &= xy - \beta z\end{aligned}$$

Gdzie:

- zmienne x , y oraz z określają aktualny stan w przestrzeni,
- parametry σ , ρ oraz β to stałe liczbowe, odpowiadające za właściwości fizyczne badanego środowiska.

Wartości tych parametrów mają kluczowe znaczenie dla zachowania układu. Najczęściej stosowane są następujące wartości: $\sigma = 10$, $\rho = 28$ oraz $\beta = 8/3$. Przy tych wartościach system wykazuje chaotyczne zachowanie [38] [20].

Po naniesieniu zmiennych na trójwymiarowy wykres przestrzeni fazowej powstaje bardzo charakterystyczny wzór. Rysowana ścieżka płynnie przeskakuje z jednej pętli na drugą, co wizualnie przypomina rozłożone skrzydła motyla. Punkt obrazujący stan układu porusza się w tej przestrzeni w nieskończoność, jednak wytyczana przez niego linia nigdy nie przecina samej siebie. Co więcej, trajektoria nigdy nie powraca dokładnie w to samo miejsce na wykresie, co oznacza, że zachowanie układu nigdy się nie powtarza [11] [25]. Stanowi to przykład atraktora dziwnego, który charakteryzuje się dużą wrażliwością na warunki początkowe. Nawet



Rysunek 4. Wykres przedstawiający atraktor Lorenza

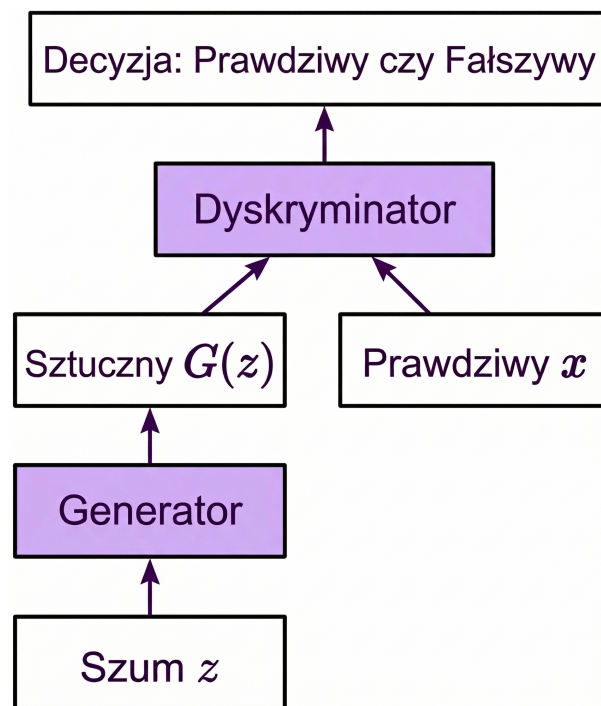
najmniejsza zmiana parametrów wejściowych powoduje, że nowa ścieżka po krótkim czasie drastycznie odbiega od poprzedniej. Na podstawie analizy takiego obrazu można łatwo określić, czy dany układ jest chaotyczny. Obecność atraktora o tak nieregularnej strukturze jest matematycznym dowodem na występowanie chaosu. Pobliskie trajektorie bardzo szybko się od siebie oddalają, ale jednocześnie pozostają uwięzione w ograniczonej przestrzeni. Dzięki temu wiadomo, że system nie dąży do prostego stanu równowagi ani nie wpada w regularny cykl. Właśnie ta kombinacja ograniczonego obszaru i ciągłego rozbiegania się ścieżek pozwala sklasyfikować układ jako w pełni chaotyczny i nieprzewidywalny w dłuższym czasie [24].

2.5 Sieci GAN

Koncepcję sieci GAN (Generative Adversarial Networks) opracowano w 2014 roku na Uniwersytecie w Montrealu w Kanadzie. Prace nad modelem prowadzono w zespole badawczym pod kierownictwem Yoshuy Bengio, a największy wkład w ten projekt przypisuje się Ianowi Goodfellowowi. Działanie tej architektury opisano szczegółowo w publikacji naukowej pod tytułem „Generative Adversarial Nets” [15].

2.5.1 Zasada działania

Architektura typu GAN składa się z dwóch współpracujących, a zarazem konkurujących ze sobą modeli sieci neuronowych: generatora oraz dyskryminatora. Stanowią one specyficzny układ, w którym obie strony wymuszają na sobie nieustanny rozwój. Proces ich uczenia przypomina grę o sumie zerowej. Oznacza to, że każdy sukces jednego modelu jest traktowany jako błąd drugiego, co zmusza je do ciągłej optymalizacji parametrów w trakcie kolejnych epok treningowych. Generator tworzy sztuczne dane w celu oszukania dyskryminatora. Z kolei dyskryminator uczy się coraz skuteczniej odróżniać fałszywe próbki od rzeczywistych [45] [14]. Ostatecznym celem tej gry jest osiągnięcie stanu równowagi, w którym generowane dane są na tyle wysokiej jakości, że stają się całkowicie nierozróżnialne od rzeczywistych sygnałów.



Rysunek 5. Schemat sieci GAN

W ujęciu matematycznym proces ten stanowi problem optymalizacji typu minimaks, co zapisuje się za pomocą funkcji wartości $V(G, D)$:

$$\min_G \max_D V(D, G) = \mathbb{E}_{x \sim p_{\text{data}}(x)} [\log D(x)] + \mathbb{E}_{z \sim p_z(z)} [\log(1 - D(G(z)))] \quad (4)$$

Gdzie:

- zmienna x oznacza dane rzeczywiste,
- zmienna z to wektor losowego szumu,
- $G(z)$ to dane wygenerowane sztucznie,
- $D(x)$ to prawdopodobieństwo, że badana próbka jest prawdziwa.

2.5.2 Generator i dyskryminator

Zadaniem generatora jest tworzenie nowych i wysoce realistycznych danych, gdzie głównym celem jest przewidzenie kolejnego punktu. Na samym początku do danych wejściowych dodawany jest szum o niewielkiej amplitudzie, zabieg ten zapobiega ciągłemu generowaniu identycznych wzorców i zapewnia odpowiednią różnorodność uzyskiwanych wyników. Działanie generatora polega na ciągłej poprawie własnych parametrów w taki sposób, aby wygenerowane wyniki były jak najtrudniejsze do odróżnienia od tych prawdziwych [12] [18].

Dyskryminator zajmuje się sprawdzaniem i oceną, dlatego na jego wejście podawane są naprzemiennie dane rzeczywiste oraz te utworzone sztucznie. Głównym jego celem jest poprawna klasyfikacja pochodzenia otrzymanej próbki. W trakcie całego procesu uczenia obie sieci aktualizują swoje parametry, co w efekcie prowadzi do stałej poprawy ich skuteczności.

Skutecznie wytrenowany układ dąży do osiągnięcia stanu pełnej równowagi, w której to generator tworzy próbki idealnie naśladujące rzeczywistość, a z kolei dyskryminator traci całkowicie zdolność odróżniania danych sztucznych od prawdziwych, co ostatecznie oznacza, że jego ocena staje się czystym zgadywaniem.

2.6 Sieci z pamięcią LSTM

Podczas analizy długich szeregów czasowych za pomocą klasycznych sieci neuronowych napotyka się trudności związane z brakiem pamięci. Zauważa się, że w takich modelach dawne informacje szybko zanikają i układ zapomina dane początkowe. Zjawisko to nazywa się zanikaniem gradientu.

Aby rozwiązać ten problem, opracowano architekturę LSTM (ang. Long Short-Term Memory). Rozwiązanie to przedstawiono w 1997 roku w publikacji naukowej autorstwa Seppa Hochreitera i Jürgena Schmidhubera pod tytułem „Long Short-Term Memory” [17]. Głównym elementem struktury LSTM jest stan komórki, oznaczany jako C_t . Traktuje się go jako ścieżkę przesyłową, po której dane przepływają przez kolejne etapy układu z bardzo małymi modyfikacjami. Pozwala to na zachowanie ważnych informacji przez długi czas [23].

2.6.1 Bramki informacyjne

Do sterowania przepływem danych wykorzystuje się specjalne mechanizmy zwane bramkami. Wykonuje się w nich operacje mnożenia macierzy i stosuje funkcje matematyczne [46] [33] [43]. Używa się funkcji sigmoidalnej σ (zwracającej wartości od 0 do 1) oraz tangensa hiperbolicznego $\tanh$ (zwracającego wartości od -1 do 1). W modelu wyróżnia się następujące elementy:

- **Brama zapominania (forget gate):** W tym miejscu ocenia się, które dane z przeszłości nie są już przydatne. Zbędne informacje usuwa się z pamięci komórki.

$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f) \quad (5)$$

- **Brama wejściowa (input gate):** Decyduje się tu, jakie nowe informacje z obecnego kroku czasowego x_t warto zapisać w systemie. Proces ten dzieli się na dwa etapy. Najpierw określa się wagę nowych danych, a później tworzy się wektor nowych wartości gotowych do zapisu $\tilde{C}_t$.

$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i) \quad (6)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C) \quad (7)$$

- **Aktualizacja stanu komórki:** W tej fazie wylicza się ostateczny, nowy stan pamięci C_t . Łączy się w nim to, co zachowano z poprzedniego kroku, z zupełnie nowymi danymi wejściowymi.

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t \quad (8)$$

- **Brama wyjściowa (output gate):** Na podstawie przefiltrowanego stanu komórki ustala się ostateczny wynik. Oblicza się stan ukryty h_t , który przekazuje się do następnego kroku czasowego i wyrzuca jako wynik pracy całej warstwy.

$$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o) \quad (9)$$

$$h_t = o_t \cdot \tanh(C_t) \quad (10)$$

2.6.2 Jednowymiarowe sieci splotowe Conv1D

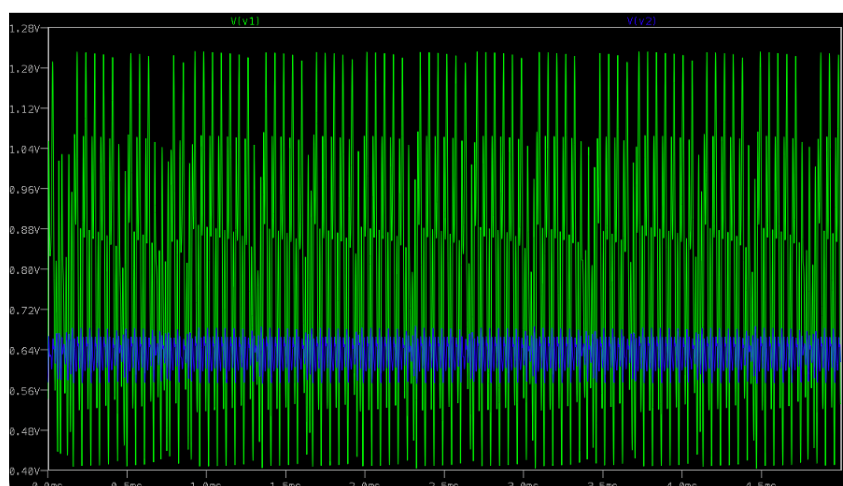
W analizie sygnałów i szeregów czasowych często stosuje się również jednowymiarowe sieci splotowe. Ich działanie polega na przesuwaniu małego filtra nad analizowanym zbiorem danych. Służy to do wykrywania lokalnych wzorców, na przykład nagłych skoków amplitudy. Takie działanie pozwala na szybkie odrzucenie szumu i przygotowanie czystszej sygnali dla warstw z pamięcią, takich jak LSTM [13] [34] [31].

Rozdział 3

Generowanie chaosu

3.1 Dane wejściowe

Jako dane wejściowe wykorzystano rzeczywisty sygnał chaotyczny, który pochodzi z symulacji układu elektronicznego opisanego w artykule naukowym pod tytułem „Evaluation of computational performance of PUF implementation for IoT device authentication” autorstwa dr. Łukasza Makowskiego. Zastosowany w badaniach układ opiera się na nieliniowym oscylatorze, który podczas przeprowadzania symulacji w środowisku LTspice wchodzi w stan chaosu deterministycznego, generując sygnał zawierający naturalny szum elektroniczny. Pozyskane w ten sposób wyniki stanowią odpowiedni materiał uczący dla badanych algorytmów, a sam analizowany zbiór danych składa się z dwóch powiązanych ze sobą przebiegów napięciowych, które oznaczono jako v1 oraz v2.



Rysunek 6. Przebiegi czasowe napięć v1 oraz v2

Przebieg przedstawia surowe dane symulacyjne wygenerowane w programie LTspice, stanowiące materiał wejściowy dla sieci GAN. Na wykresie zaprezentowano przebiegi czasowe wspomnianych sygnałów napięciowych, w których wyraźnie widać ich oscylacyjną i silnie nie-

regularną naturę. Przedstawione fale charakteryzują się całkowitym brakiem okresowości oraz ciągłymi, nieprzewidywalnymi zmianami amplitudy, co stanowi bezpośrednie odzwierciedlenie zjawiska chaosu deterministycznego panującego w obwodzie.

3.2 Kod źródłowy

Poniżej zamieszczono kod źródłowy, który implementuje architekturę sieci GAN i służy do przeprowadzenia procesu uczenia na podstawie dostarczonych danych wejściowych.

3.2.1 Importowanie bibliotek

Na samym początku zaimportowana potrzebne biblioteki, takie jak `numpy` i `pandas` potrzebne do analizy danych numerycznych oraz moduł `pyplot` do tworzenia wykresów. Do budowy modeli uczenia maszynowego wykorzystano środowisku `tensorflow`, skąd zaimportowano konkretne warstwy sieci neuronowej, takie jak `LSTM`, `Dense` czy `Conv1D`. Oprócz tego zastosowano narzędzie `MinMaxScaler` z biblioteki `scikit-learn`, aby odpowiednio znormalizować wprowadzane dane wejściowe.

```
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4 import os
5 import datetime
6 import tensorflow as tf
7 from tensorflow.keras.models import Model
8 from tensorflow.keras.layers import LSTM, Dense, Input, LeakyReLU,
9     Concatenate, GaussianNoise, Dropout, Conv1D
10 from tensorflow.keras.optimizers import Adam
11 from sklearn.preprocessing import MinMaxScaler
```

3.2.2 Inicjalizacja klasy głównej

Zdefiniowano główną klasę programu `SystemGAN`, która zarządza modelem GAN. Wskazano folder do zapisu wyników, a w przypadku jego braku następuje automatyczne utworzenie nowego katalogu. Następnie określono wartości podstawowych hiperparametrów, takich jak długość sekwencji, liczba kroków, ilość epok oraz wielkość paczki przetwarzanej w jednym kroku. Skonfigurowano również narzędzie `MinMaxScaler`, aby skalowało dane wejściowe do przedziału od -1 do 1. Na koniec wywołano funkcje budujące struktury generatora i dyskryminatora, a dla obu sieci przygotowano algorytmy optymalizacji `Adam` z odpowiednimi współczynnikami uczenia.

```
1 class SystemGAN:
```

```
2 def __init__(self):
3     self.folder_name = "Wyniki_GAN"
4     if not os.path.exists(self.folder_name):
5         os.makedirs(self.folder_name)
6
7     self.seq_len = 64
8     self.predict_steps = 2000
9     self.epochs = 150
10    self.batch_size = 32
11    self.timestep = 2.5e-6
12    self.dt_eff = self.timestep
13
14    self.scaler = MinMaxScaler(feature_range=(-1, 1))
15
16    # self.analizator = AnalizaChaosu(self.folder_name)
17    self.generator = self.buduj_generator()
18    self.discriminator = self.buduj_dyskryminator()
19
20    self.g_optimizer = Adam(learning_rate=0.0002, beta_1=0.5)
21    self.d_optimizer = Adam(learning_rate=0.0001, beta_1=0.5)
```

3.2.3 Przygotowanie Danych

Funkcja wczytuje zbiór danych z pliku w formacie CSV i poddaje go wstępnej obróbce, gdzie po sprawdzeniu poprawności struktury pliku zastosowano mechanizm redukcji próbek polegający na wybieraniu co piątej wartości w przypadku obszernych zbiorów danych w celu zmniejszenia kosztów obliczeniowych. Kolejnym etapem jest normalizacja wyodrębnionego sygnału przy pomocy wyuczonego skalara, sprowadzającego wszystkie wyniki do ujednoliconego i mniejszego przedziału na podstawie wcześniej zapamiętanych wartości skrajnych. Na samym końcu utworzono zbiory uczące poprzez układanie danych w okna przesuwne, dzięki czemu zmienna wyjściowa x zawiera gotowe sekwencje uczące, natomiast zmienna y odpowiada wartościom przewidywanym tuż po zakończeniu każdej z nich.

```
1 def wczytaj_dane(self, nazwa_pliku):
2     print("Wczytywanie danych...")
3     try:
4         df = pd.read_csv(nazwa_pliku, header=None)
5         data_len = len(df)
6
7         if df.shape[1] < 3:
8             raise ValueError(f"Plik {nazwa_pliku} musi mieć co najmniej 3 kolumny (np. indeks, x, y).")
9
10        if data_len < 5000:
11            downsample = 1
```

```
12 print("Maly zbior danych - pobieranie wszystkich probek.")
13 else:
14     downsample = 5
15     print(f"Duzy zbior danych - pobieranie co {downsample} probki.")
16
17     self.signal_raw = df.iloc[:,downsample, 1:3].values
18     self.dt_eff = self.timestep * downsample
19
20     except FileNotFoundError:
21         print(f"Blad: brak pliku {nazwa_pliku}")
22         raise
23     except ValueError as e:
24         print(f"Blad wczytywania danych: {e}")
25         raise
26
27     signal_scaled = self.scaler.fit_transform(self.signal_raw)
28
29     X, y = [], []
30     for i in range(len(signal_scaled) - self.seq_len):
31         X.append(signal_scaled[i : i + self.seq_len])
32         y.append(signal_scaled[i + self.seq_len])
33
34     self.X = np.array(X).astype(np.float32)
35     self.y = np.array(y).astype(np.float32)
36
37     if len(self.X) < self.batch_size:
38         self.batch_size = len(self.X)
```

3.2.4 Struktura Generatora

Funkcja odpowiada za utworzenie architektury generatora, którego głównym zadaniem jest płynne i wiarygodne tworzenie nowych wartości na podstawie ich historii sygnału. Proces przetwarzania danych zaczyna się od dodania do sygnału wejściowego niewielkiego szumu Gaussa. Zabieg ten wymusza na sieci szukanie ogólnych wzorców, zwiększając stabilność treningu i zapobiegając zjawisku przeuczenia. Następnie dane przekazano do warstwy splotowej `Conv1D`, która ma za zadanie wyodrębnić z sygnału charakterystyczne wahania, po czym zastosowano dwie warstwy z komórkami pamięci `LSTM` służące do analizowania ogólnych tendencji wahań w dłuższym przedziale czasowym. Pomiedzy wymienionymi warstwami wykorzystano funkcję aktywacji `LeakyReLU` oraz mechanizm porzucania `Dropout` chroniący przed nadmiernym dopasowaniem modelu do danych treningowych, a na końcu utworzono warstwę wyjściową z dwoma neuronami i funkcją aktywacji typu tangens hiperboliczny, co ostatecznie gwarantuje utrzymanie wyników w pożądanym przedziale wartości.

```
1 def buduj_generator(self):
```

```

2 inp = Input(shape=(self.seq_len, 2))
3 x = GaussianNoise(0.05)(inp)
4
5 x = Conv1D(filters=64, kernel_size=5, padding='same')(x)
6 x = LeakyReLU(negative_slope=0.2)(x)
7
8 x = LSTM(128, return_sequences=True)(x)
9 x = LeakyReLU(negative_slope=0.2)(x)
10 x = Dropout(0.2)(x)
11
12 x = LSTM(64)(x)
13 x = LeakyReLU(negative_slope=0.2)(x)
14
15 out = Dense(2, activation='tanh')(x)
16 return Model(inp, out)

```

3.2.5 Struktura Dyskryminatora

Scalone dane przekazuje się następnie do ukrytej warstwy gęstej składającej się z 32 neuronów w celu zbadania logicznych powiązań, by na samym końcu wykorzystać warstwę wyjściową z jednym neuronem o sigmoidalnej funkcji aktywacji. Dzięki takiemu zabiegowi ostateczny wynik zawsze mieści się w przedziale od zera do jeden, precyzyjnie określając prawdopodobieństwo pochodzenia ocenianego punktu z prawdziwego zbioru danych. Zdefiniowano architekturę dyskryminatora posiadającego dwa odrębne wejścia, z których pierwsze przyjmuje historyczną sekwencję danych poddawaną początkowej analizie w warstwie LSTM, natomiast drugie pobiera pojedynczy, nowy punkt w czasie. Wprowadzona historia sygnału trafia do warstwy wstępnej wyposażonej w 64 neurony z funkcją aktywacji zapobiegającą wygasaniu układu, po czym przetworzona historia oraz sprawdzany punkt są łączone w jeden wspólny wektor informacji. Scalone dane przekazuje się następnie do ukrytej warstwy gęstej składającej się z 32 neuronów. W strukturze tej każdy pojedynczy neuron jest bezpośrednio połączony ze wszystkimi elementami warstwy poprzedzającej, co umożliwia bardzo dokładne badanie skomplikowanych powiązań logicznych wewnątrz modelu. Na samym końcu wykorzystano warstwę wyjściową z pojedynczym neuronem o sigmoidalnej funkcji aktywacji, która pełni rolę specjalnego filtra matematycznego ujednoliconego dane. Dzięki temu ostateczny wynik zawsze mieści się w ścisłym przedziale od 0 do 1.

```

1 def buduj_dyskryminator(self):
2     inp_seq = Input(shape=(self.seq_len, 2))
3     inp_next = Input(shape=(2,))
4     x = LSTM(64)(inp_seq)
5     x = LeakyReLU(negative_slope=0.2)(x)
6     combined = Concatenate()([x, inp_next])
7     c = Dense(32)(combined)

```

```
8 c = LeakyReLU(negative_slope=0.2)(c)
9 out = Dense(1, activation='sigmoid')(c)
10 return Model([inp_seq, inp_next], out)
```

3.2.6 Procedura Treningowa

Zaprezentowany fragment realizuje pojedynczy krok uczenia z wykorzystaniem algorytmu propagacji wstecznej. Działanie to można przyrównać do kontroli jakości na linii produkcyjnej. Najpierw generowany jest ostateczny wynik, a następnie oblicza się wielkość pomyłki w stosunku do założeń idealnych. Aby zapobiec podobnym błędom w przyszłości, informacja o tej pomyłce cofana jest krok po kroku od ostatniego etapu obliczeń aż do samego początku systemu. Taki powrót sygnału pozwala precyzyjnie ustalić, które wewnętrzne parametry matematyczne wymagają poprawy. Do optymalizacji działania zastosowano dekorator `@tf.function`, który zamienia kod na wydajny graf obliczeniowy. W tym podejściu skomplikowane równania dzieli się na mapę prostych zadań. Przed startem obliczeń analizowana jest cała mapa, co pozwala na zignorowanie niepotrzebnych etapów i zgrupowanie działań niezależnych od siebie. Zgrupowane zadania przesyłane są następnie do karty graficznej, gdzie rozwiązuje się je w tym samym czasie, co powoduje ogromną oszczędność zasobów.

Rejestracja operacji matematycznych odbywa się przy pomocy klasy `GradientTape`. Umożliwia to wyznaczenie pochodnych i aktualizację wag dyskryminatora na podstawie błędów w ocenie próbek prawdziwych oraz wygenerowanych. Po zakończeniu tego etapu rozpoczyna się proces uczenia generatora. Obliczana dla niego funkcja błędu składa się z trzech elementów: karę za zdemaskowanie przez dyskryminator, błąd bezwzględny z przypisaną wagą 100 oraz składnik regularyzacyjny. Ten ostatni element pełni funkcję stabilizującą, nakłada on karę matematyczną za zbyt gwałtowne skoki wartości między nowo tworzonym punktem a końcem aktualnej sekwencji.

```
1 @tf.function
2 def krok_treningu(self, real_seq, real_next):
3     with tf.GradientTape() as tape_d:
4         generated_next = self.generator(real_seq, training=True)
5         pred_real = self.discriminator([real_seq, real_next], training=True)
6         pred_fake = self.discriminator([real_seq, generated_next], training=True)
7         d_loss = tf.reduce_mean(tf.keras.losses.binary_crossentropy(tf.ones_like(
8             pred_real), pred_real) +
9             tf.keras.losses.binary_crossentropy(tf.zeros_like(pred_fake), pred_fake))
10
11         grads_d = tape_d.gradient(d_loss, self.discriminator.trainable_variables)
12         self.d_optimizer.apply_gradients(zip(grads_d, self.discriminator.
13             trainable_variables))
14
15     with tf.GradientTape() as tape_g:
```

```

14 generated_next = self.generator(real_seq, training=True)
15 pred_fake = self.discriminator([real_seq, generated_next], training=True)
16
17 g_adv_loss = tf.keras.losses.binary_crossentropy(tf.ones_like(pred_fake),
18     pred_fake)
19
20 g_mae_loss = tf.reduce_mean(tf.abs(real_next - generated_next))
21
22 ostatni_punkt_sekwencji = real_seq[:, -1, :]
23 kara_za_skoki = tf.reduce_mean(tf.square(generated_next -
24     ostatni_punkt_sekwencji))
25 total_g_loss = tf.reduce_mean(g_adv_loss) + 100.0 * g_mae_loss + 1.0 *
26     kara_za_skoki
27
28 grads_g = tape_g.gradient(total_g_loss, self.generator.trainable_variables)
29 self.g_optimizer.apply_gradients(zip(grads_g, self.generator.
30     trainable_variables))
31
32 return d_loss, total_g_loss

```

3.2.7 Pętla Ucząca

Zaimplementowano główną pętlę procesu uczenia maszynowego, w której skonstruowano zoptymalizowany strumień danych za pomocą interfejsu `tf.data.Dataset`, co zapewnia poprawne przetasowanie paczek danych przed każdą epoką. Zdefiniowana wcześniej metoda wykonująca krok trenowania uruchamiana jest dla każdego zestawu danych wejściowych, a obliczone wartości funkcji strat są systematycznie gromadzone i wyświetlane na ekranie co dziesięć epok, co pozwala na ciągłe nadzorowanie zachowania modelu pod kątem występowania ewentualnych nieprawidłowości.

```

1 def trenuj(self):
2     print("Start treningu...")
3     dataset = tf.data.Dataset.from_tensor_slices((self.X, self.y)).shuffle
4         (1000).batch(self.batch_size, drop_remainder=True)
5
6     for epoch in range(self.epochs):
7         d_losses, g_losses = [], []
8         for seq, nxt in dataset:
9             d, g = self.krok_treningu(seq, nxt)
10            d_losses.append(d)
11            g_losses.append(g)
12        if (epoch+1) % 10 == 0:
13            print(f"Epoka {epoch+1}/{self.epochs} | D {np.mean(d_losses):.4f} | G {np.
14                mean(g_losses):.4f}")

```

3.2.8 Generowanie przebiegów

Sekcja ta ma za zadanie wygenerowanie danych w oparciu o wyuczony już model sieci, wykorzystując do tego tryb iteracyjny, w którym każda nowa próbka sygnału przewidziana przez generator jest dopisywana na koniec okna sekwencji w miejsce jej najstarszego elementu. Procedura ta powtarza się przez ustaloną liczbę kroków wybiegających w przyszłość, po czym na wygenerowanym sztucznie przebiegu wywołuje się funkcję skalera w celu odwrócenia wcześniej wykonanej normalizacji.

```
1 def generuj(self):
2     print("Generowanie przebiegu...")
3     ts = datetime.datetime.now().strftime("%H%M%S")
4
5     idx_start = 0
6     if len(self.X) > self.predict_steps:
7         idx_start = len(self.X) - self.predict_steps - 1
8
9     curr_seq = self.X[idx_start].reshape(1, self.seq_len, 2)
10    history_seq = curr_seq.copy()
11
12    preds = []
13    for i in range(self.predict_steps):
14        next_point = self.generator(curr_seq, training=False).numpy()
15        preds.append(next_point[0])
16        curr_seq = np.concatenate([curr_seq[:, 1:, :], next_point.reshape(1, 1, 2)
17                                   ], axis=1)
18    if i % 100 == 0:
19        print(f"Krok {i}/{self.predict_steps}")
```

3.3 Uzasadnienie wyboru architektury GAN

W analizowanym rozwiązaniu zdecydowano się na wykorzystanie architektury generatywnych sieci przeciwstawnych, ponieważ badane układy dynamiczne charakteryzują się ogromną wrażliwością nawet na najdrobniejsze zmiany warunków początkowych. Tradycyjne modele uczenia maszynowego poprzez minimalizację błędu średniokwadratowego mają silną tendencję do uśredniania wyników podczas prognozowania wielokrokowego, co skutkuje szybkim zanikiem naturalnej dynamiki badanego sygnału i generowaniem płaskiej linii dążącej do średniej wartości zbioru. Zastosowana architektura z łatwością eliminuje ten problem, gdyż dzięki wbudowanej obecności modelu dyskryminatora cały układ musi nie tylko wyznaczyć konkretną wartość liczbową, ale również stworzyć próbkę przypominającą swoim zachowaniem i dynamiką autentyczny fragment sygnału. Dyskryminator wymusza w ten sposób na generatorze ciągłe utrzymywanie charakterystycznego kształtu atraktora oraz naturalnej zmienności przebiegu,

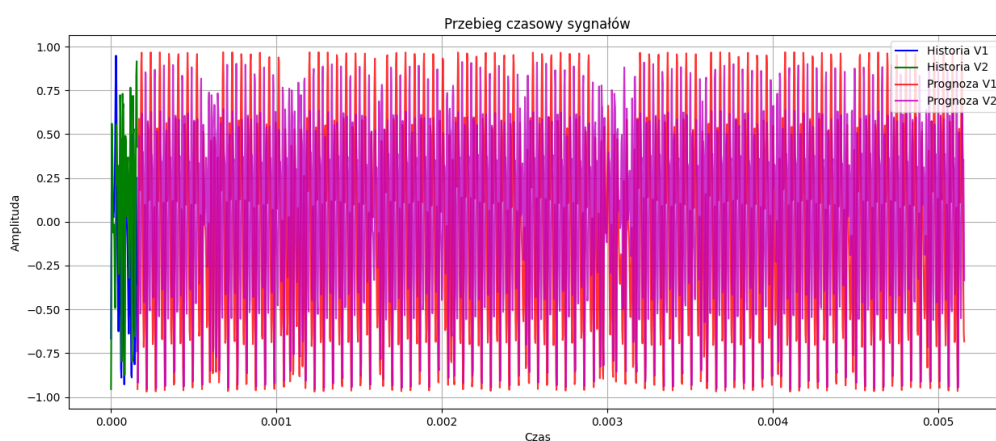
skutecznie zapobiegając rozmywaniu się predykcji, co stanowi absolutnie kluczowy aspekt przy poprawnym modelowaniu zjawiska chaosu deterministycznego.

Rozdział 4

Dowód chaosu

4.1 Analiza Chaosu

Po wygenerowaniu przebiegu przez sieć GAN konieczne jest przeprowadzenie szczegółowej analizy jego właściwości chaotycznych. W tym celu zaprojektowano specjalną klasę, której głównym zadaniem jest ocena jakości wygenerowanych danych poprzez obliczenie podstawowych wskaźników chaosu deterministycznego, takich jak największy wykładnik Lapunowa oraz znormalizowana entropia Shannona. Dodatkowo w ramach tej klasy zaimplementowano funkcje umożliwiające wizualizację atraktora Lorenza oraz badanie korelacji pomiędzy kolejnymi fragmentami sygnału, co pozwala na zncznie głębsze zrozumienie struktury wygenerowanego przebiegu.



Rysunek 7. Przebieg czasowy sygnału chaotycznego

Na wykresie zaprezentowano fragment rzeczywistego sygnału chaotycznego (kolor zielony i niebieski), który posłużył jako materiał uczący dla sieci GAN. Pozostała część widocznego sygnału (kolor czerwony i różowy) to wygenerowany przez sieć przebieg, który stanowi przedmiot dalszej analizy. Oba sygnały charakteryzują się nieregularną, oscylacyjną naturą, co

jest typowe dla układów chaotycznych. Widać, że wygenerowany przebieg zachowuje podobne cechy do oryginalnego, co sugeruje, że sieć GAN była w stanie nauczyć się podstawowych wzorców dynamiki układu, choć ostateczna ocena jakości wygenerowanych danych będzie opierać się na bardziej szczegółowych wskaźnikach.

4.2 Wykładnik Lapunowa

Zaprojektowano metodę obliczającą największy wykładnik Lapunowa w oparciu o algorytm Rosensteina, który znajduje szerokie zastosowanie podczas analizowania sygnałów chaotycznych. Algorytm na samym początku losuje reprezentatywną próbę punktów odniesienia z badanej trajektorii fazowej, a następnie dla każdego z nich poszukuje najbliższego sąsiada w przestrzeni, dbając jednocześnie o zachowanie odpowiedniej separacji czasowej w celu uniknięcia fałszywych korelacji. W kolejnym etapie ślędzona jest ewolucja obu początkowo bliskich sobie punktów i rejestrowane jest ich wzajemne oddalanie się w kolejnych krokach iteracyjnych. Zgromadzone wartości logarytmów z obliczonych odległości są uśredniane dla każdej iteracji, po czym wyodrębnia się początkowy obszar liniowego narastania błędu i dopasowuje do niego prostą. Współczynnik kierunkowy wyznaczonej prostej po podzieleniu przez zastosowany krok całkowania bezpośrednio definiuje ostateczną wartość wykładnika Lapunowa.

```
1  def lapunow(self, signal, emb_dim=3, tau=1, min_tsep=10, max_k=50):
2      N = len(signal)
3      N_emb = N - (emb_dim - 1) * tau
4      X = np.array([signal[i : i + emb_dim*tau : tau] for i in range(
      N_emb)])
5
6      nn_indices = []
7      for i in range(N_emb):
8          dists = np.max(np.abs(X - X[i]), axis=1)
9          dists[max(0, i-min_tsep):min(N_emb, i+min_tsep+1)] = np.inf
10         nn = np.argmin(dists)
11         nn_indices.append(nn)
12
13         S = np.zeros(max_k)
14         c = np.zeros(max_k)
15
16         for k in range(max_k):
17             for i, nn in enumerate(nn_indices):
18                 if i + k < N_emb and nn + k < N_emb:
19                     dist = np.max(np.abs(X[i+k] - X[nn+k]))
20                     if dist > 0:
21                         S[k] += np.log(dist)
22                         c[k] += 1
23
```

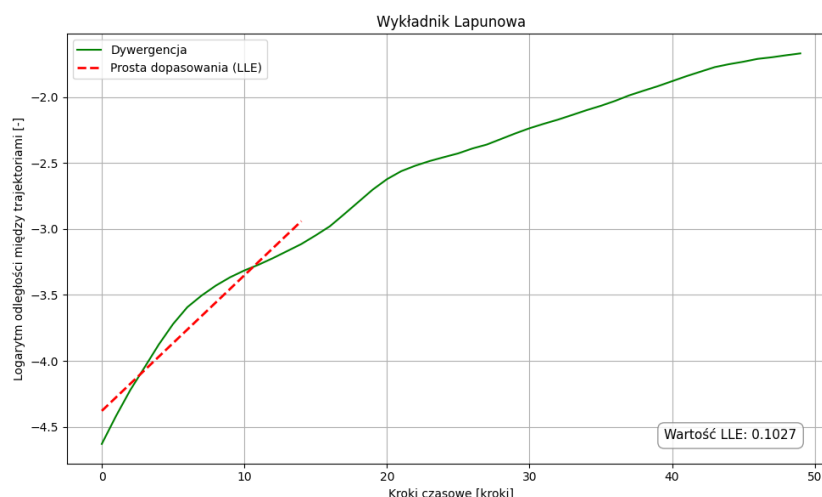


```

24 with np.errstate(divide='ignore', invalid='ignore'):
25     return S / c

```

4.2.1 Analiza wyników



Rysunek 8. Wykładnik Lapunowa

Wykładnik Lapunowa ma wartość dodatnią, co jednoznacznie wskazuje na obecność chaosu deterministycznego w wygenerowanym sygnale. Wartość ta wynosi powyżej 0.1, co jest typowe dla układów chaotycznych i sugeruje, że trajektoria fazowa charakteryzuje się silną wrażliwością na warunki początkowe. Otrzymany wynik potwierdza, że sieć GAN była w stanie nauczyć się generować dane, które zachowują kluczowe właściwości dynamiki układu, a wygenerowany przebieg posiada cechy charakterystyczne dla zjawisk nieliniowych. Dodatnia wartość wykładnika Lapunowa jednoznacznie wskazuje, że wygenerowany sygnał jest trudny do przewidzenia w dłuższym horyzoncie czasowym, co jest typowe dla układów chaotycznych.

4.3 Analiza entropii

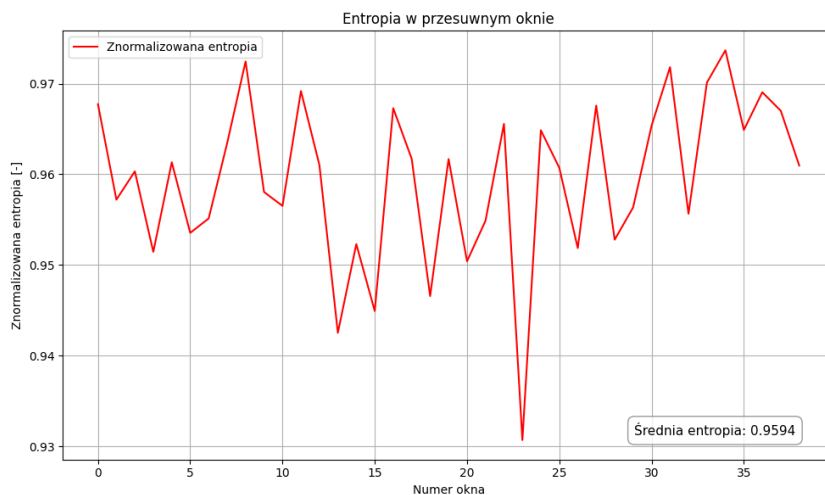
Zaimplementowano metodę analizy statystycznej wygenerowanego sygnału wykorzystującą przesuwne okno pomiarowe poruszające się wzdłuż badanej osi czasu. Zdefiniowano w niej wewnętrzną funkcję obliczającą znormalizowaną entropię informacyjną Shannona na podstawie histogramu rozkładu wartości w danym wycinku. Zastosowana normalizacja polega na podzieleniu klasycznego wyniku przez maksymalną wartość, co pozwala na sprowadzenie odczytów do bardzo wygodnego zakresu od 0 do 1. Otrzymane z pętli wyniki cząstkowe są następnie gromadzone w jednej liście, a ich ostateczna średnia arytmetyczna zostaje wyliczona i zaprezentowana użytkownikowi w czytelnej ramce.

```

1  def analiza_entropii_i_korelacji(self, generowany, ts):
2      print("Analiza entropii i korelacji...")
3      rozmiar_okna = 100
4      skok = 50
5      liczba_koszy = 20
6
7      def licz_znormalizowana_entropie(fragment):
8          hist, _ = np.histogram(fragment, bins=liczba_koszy)
9          p = hist / np.sum(hist)
10         p = p[p > 0]
11         entropia = -np.sum(p * np.log2(p))
12         entropia_maksymalna = np.log2(liczba_koszy)
13         return entropia / entropia_maksymalna if entropia_maksymalna >
0 else 0
14
15     entropia_gen = []
16     for i in range(0, len(generowany) - rozmiar_okna + 1, skok):
17         frag_gen = generowany[i : i + rozmiar_okna, 0]
18         entropia_gen.append(licz_znormalizowana_entropie(frag_gen))

```

4.3.1 Analiza wyników



Rysunek 9. Entropia

Znormalizowana entropia informacyjna Shannona dla wygenerowanego sygnału wynosi około 0.96, co wykazuje znaczny stopień nieuporządkowania i wewnętrznej złożoności charakterystyczny dla układów chaotycznych. Przyjmuje się powszechnie, że stan chaosu deterministycznego można z powodzeniem określić w sytuacji, gdy znormalizowana miara Shannona przyjmuje wartości przekraczające próg 0.6. Warunek ten został w tym konkretnym przypadku

spełniony, a zatem można stanowczo stwierdzić, że wygenerowany przebieg posiada silne cechy charakterystyczne dla zjawisk nieliniowych. Wysoka wartość entropii jednoznacznie sugeruje, że sztuczny szereg czasowy zawiera dużą ilość informacji i jest niezwykle trudny do przewidzenia w dłuższych badaniach. Analiza tego wskaźnika pozwala na rzetelną ocenę jakości danych opracowanych przez sieć neuronową.

4.4 Badanie korelacji

W drugiej części badanej metody zaprojektowano algorytm przeznaczony do wyznaczania współczynników korelacji pomiędzy odrębnymi wycinkami zebranego przebiegu sztucznego. Zastosowany mechanizm systematycznie przesuwając okno wzdłuż osi czasu i dokonując precyzyjnych obliczeń matematycznego podobieństwa pomiędzy bieżącym fragmentem sygnału a fragmentem wygenerowanym bezpośrednio przed nim. Procedura ta pozwala na niezwykle skuteczną weryfikację stabilności badanego systemu poprzez oznaczanie autokorelacji krótko-czasowej dwóch sąsiadujących ze sobą ram pomiarowych.

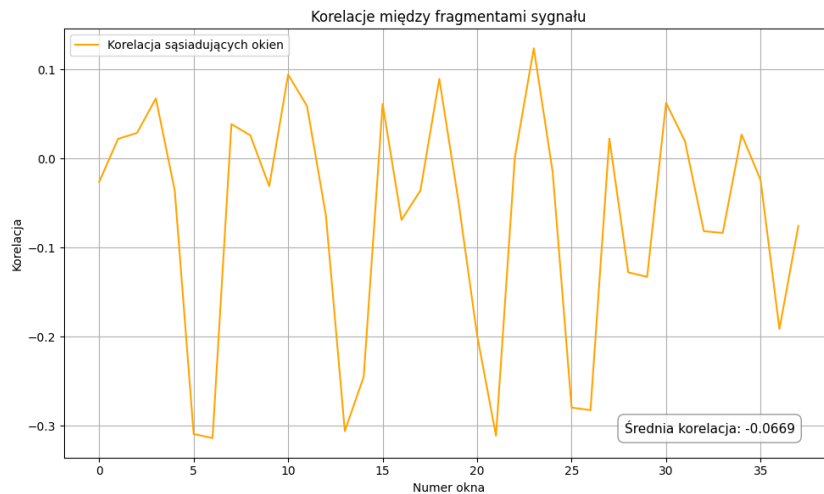
```

1      def bezpieczna_korelacja(a, b):
2          if np.std(a) == 0 or np.std(b) == 0:
3              return 0
4          return np.corrcoef(a, b)[0, 1]
5
6      korelacje_gen_gen = []
7      poprzedni_gen = generowany[0 : rozmiar_okna, 0]
8
9      for i in range(skok, len(generowany) - rozmiar_okna + 1, skok):
10         obecny_gen = generowany[i : i + rozmiar_okna, 0]
11         korelacje_gen_gen.append(bezpieczna_korelacja(obecny_gen,
12             poprzedni_gen))
13         poprzedni_gen = obecny_gen

```

4.4.1 Analiza wyników

Na powyższym wykresie przedstawiono współczynniki korelacji wyliczone pomiędzy kolejnymi, następującymi po sobie fragmentami wygenerowanego sygnału. Wartości te nieustannie oscylują w bliskich okolicach zera, co wyraźnie dowodzi brak zależności liniowych. Niskie wartości autokorelacji wskazują na ogromną zmienność i nieregularność w strukturze przesuwanego okna, co idealnie wpisuje się w oczekiwania stawiane przed układami chaotycznymi. Zebrane w ten sposób dane pozwalają na ocenę stopnia niezależności poszczególnych ramek sygnału oraz na wykluczenie obecności niepożądanych, deterministycznych oscylacji okresowych w wygenerowanym przez sieć przebiegu.



Rysunek 10. Korelacje

4.5 Atraktor Lorenza

Ten fragment kodu jest odpowiedzialny za wizualizację atraktora Lorenza. Funkcja przyjmuje jako argumenty wejściowe macierz z przewidywanymi wartościami oraz znacznik czasowy wykorzystywany do poprawnego nazwania pliku. Wykorzystując zewnętrzną bibliotekę graficzną, zaimplementowany skrypt tworzy przejrzysty wykres dwuwymiarowej przestrzeni fazowej. Po zakończeniu procesu nakładania punktów, gotowy obraz zapisywany jest w wyznaczonym folderze głównym systemu.

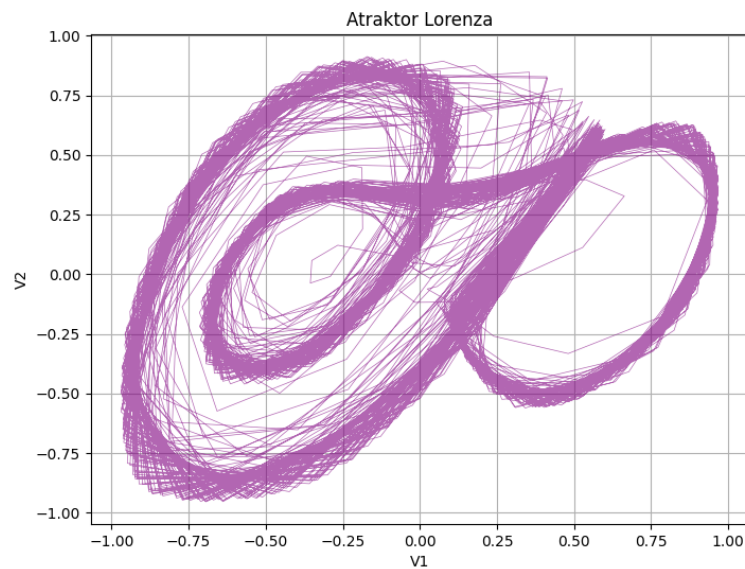
Ten fragment kodu po prostu generuje ostateczny rysunek atraktora. Najpierw przygotowuje się odpowiednio duże tło, a następnie nanosi się na nie przewidziane przez sieć punkty. Łączy się je ze sobą cieniutką, fioletową i lekko przezroczystą linią, dzięki czemu powstaje charakterystyczny kształt przypominający skrzydła motyla. Na sam koniec dodaje się tylko niezbędny tytuł, podpisy osi oraz siatkę pomocniczą, po czym gotowy obrazek zostaje od razu zapisany w odpowiednim folderze na dysku komputera.

```

1  def rysuj_atraktor(self, generowany, ts):
2      print("Rysowanie atraktora...")
3      plt.figure(figsize=(8, 6))
4      plt.plot(generowany[:, 0], generowany[:, 1], color='purple', alpha
=0.6, lw=0.5)
5      plt.title("Atraktor Lorenza")
6      plt.xlabel("V1")
7      plt.ylabel("V2")
8      plt.grid(True)
9      plt.savefig(os.path.join(self.folder_name, f"Atraktor_{ts}.png"))
10     plt.close()

```

4.5.1 Analiza wyników



Rysunek 11. Atraktor Lorenza

Na powyższym wykresie zaprezentowano przestrzeń fazową atraktora Lorenza wygenerowaną z bardzo dużą dokładnością na podstawie danych stworzonych przez wytrenowaną sieć neuronową. Charakterystyczna struktura przypominająca rozłożone skrzydła motyla jest na tym rysunku wyraźnie widoczna, co jednoznacznie potwierdza prawidłowe uchwycenie kluczowych właściwości badanej dynamiki przez nieliniowy model matematyczny. Dzięki uzyskanej wizualizacji można w prosty sposób zaobserwować typowe, nieregularne oscylacje charakterystyczne dla przebiegu chaotycznego.

Rozdział 5

Podsumowanie

Bibliografia

- [1] Abarbanel, H. D., Brown, R. i Kennel, M. B., „Local Lyapunov exponents computed from observed data”, *Journal of Nonlinear Science*, t. 2, nr. 3, s. 343–365, 1992.
- [2] Andrzej Czajka, A. G. P., „Złożoność danych pomiarowych i metody jej określania”, *Przegląd Elektrotechniczny*,
- [3] Benesty, J., Chen, J., Huang, Y. i Cohen, I., „Pearson correlation coefficient”, w *Noise reduction in speech processing*, Springer, 2009, s. 1–4.
- [4] Borowska, M., *Wprowadzenie do zastosowania entropii w analizie sygnałów i obrazów biomedycznych oraz jej aplikacje w medycynie i weterynarii*. Politechnika Białostocka, 2023.
- [5] Bromiley, P., Thacker, N. i Bouhova-Thacker, E., „Shannon entropy, Renyi entropy, and information”, *Statistics and Inf. Series (2004-004)*, t. 9, nr. 2004, s. 2–8, 2004.
- [6] De Souza, S. L. i Caldas, I. L., „Calculation of Lyapunov exponents in systems with impacts”, *Chaos, Solitons & Fractals*, t. 19, nr. 3, s. 569–579, 2004.
- [7] Dingwell, J. B., „Lyapunov exponents”, *Wiley encyclopedia of biomedical engineering*, 2006.
- [8] Dufera, A. G., Liu, T. i Xu, J., „Regression models of Pearson correlation coefficient”, *Statistical Theory and Related Fields*, t. 7, nr. 2, s. 97–106, 2023.
- [9] Ellerman, D., *An introduction to logical entropy and its relation to Shannon entropy*, 2013.
- [10] Eskov, V., Eskov, V., Vochmina, Y. V., Gorbunov, D. i Ilyashenko, L., „Shannon entropy in the research on stationary regimes and the evolution of complexity”, *Moscow university physics bulletin*, t. 72, nr. 3, s. 309–317, 2017.
- [11] Galias, Z. i Zgliczyński, P., „Computer assisted proof of chaos in the Lorenz equations”, *Physica D: Nonlinear Phenomena*, t. 115, nr. 3-4, s. 165–188, 1998.
- [12] Goodfellow, I., „Nips 2016 tutorial: Generative adversarial networks”, *arXiv preprint arXiv:1701.00160*, 2016.
- [13] Goodfellow, I., Bengio, Y., Courville, A. i Bengio, Y., *Deep learning*. MIT press Cambridge, 2016, t. 1.
- [14] Goodfellow, I. i in., „Generative adversarial networks”, *Communications of the ACM*, t. 63, nr. 11, s. 139–144, 2020.

- [15] Goodfellow, I. J. i in., „Generative adversarial nets”, *Advances in neural information processing systems*, t. 27, 2014.
- [16] Hallmann, D. i Jankowski, P., „Badanie przydatności procedur Rosensteina i Eckmanna do identyfikacji chaotycznych szeregów czasowych”, *Zeszyty Naukowe Akademii Morskiej w Gdyni*, nr. 103, s. 120–136, 2018.
- [17] Hochreiter, S. i Schmidhuber, J., „Long short-term memory”, *Neural computation*, t. 9, nr. 8, s. 1735–1780, 1997.
- [18] Jiang, F., Ma, J., Webster, C. J., Li, X. i Gan, V. J., „Building layout generation using site-embedded GAN model”, *Automation in construction*, t. 151, s. 104888, 2023.
- [19] Lorenz, E., „The butterfly effect”, w *The chaos avant-garde: Memories of the early days of chaos theory*, World Scientific, 2000, s. 91–94.
- [20] Lorenz, E. N. i Haman, K., „The essence of chaos”, *Pure and Applied Geophysics*, t. 147, nr. 3, s. 598–599, 1996.
- [21] Ly, A., Marsman, M. i Wagenmakers, E.-J., „Analytic posteriors for Pearson’s correlation coefficient”, *Statistica Neerlandica*, t. 72, nr. 1, s. 4–13, 2018.
- [22] Lyapunov, A. M., „The general problem of the stability of motion”, *International journal of control*, t. 55, nr. 3, s. 531–534, 1992.
- [23] Madondo, M. i Gibbons, T., „Learning and modeling chaos using lstm recurrent neural networks”, w *Proceedings of the midwest instruction and computing symposium, duluth, minnesota*, 2018, s. 6–7.
- [24] Mischaikow, K. i Mrozek, M., „Chaos in the Lorenz equations: a computer-assisted proof”, *Bulletin of the American Mathematical Society*, t. 32, nr. 1, s. 66–72, 1995.
- [25] Mischaikow, K., Mrozek, M. i Szymczak, A., „Chaos in the lorenz equations: A computer assisted proof part iii: Classical parameter values”, *Journal of Differential Equations*, t. 169, nr. 1, s. 17–56, 2001.
- [26] Miśkiewicz, M., „Zastosowanie wykładników Lapunowa do prognozowania zjawisk ekonomicznych opisanych za pomocą szeregów czasowych”, *Prace naukowe Akademii Ekonomicznej we Wrocławiu. Zastosowanie Metod Ilościowych*, t. 1189, s. 211–223, 2007.
- [27] Miśkiewicz-Nawrocka, M., „Wpływ redukcji szumu losowego metodą najbliższych sąsiadów na wyniki prognoz otrzymanych za pomocą największego wykładnika Lapunowa”, *Studia Ekonomiczne*, nr. 159, s. 82–98, 2013.
- [28] Miśkiewicz-Nawrocka, M., „Wpływ redukcji szumu losowego metodą najbliższych sąsiadów na stabilność największego wykładnika Lapunowa w ekonomicznych szeregach czasowych”, *Studia Ekonomiczne*, t. 203, s. 101–113, 2014.
- [29] Obilor, E. I. i Amadi, E. C., „Test for significance of Pearson’s correlation coefficient”, *International Journal of Innovative Mathematics, Statistics & Energy Policies*, t. 6, nr. 1, s. 11–23, 2018.

-
- [30] Parandowski, J., *Mitologia*. Puls, 1992.
- [31] Phiphitphatphaisit, S. i Surinta, O., „Deep feature extraction technique based on Conv1D and LSTM network for food image recognition”, *Engineering and Applied Science Research*, t. 48, nr. 5, s. 581–592, 2021.
- [32] Rajaram, R., Castellani, B. i Wilson, A., „Advancing Shannon entropy for measuring diversity in systems”, *Complexity*, t. 2017, nr. 1, s. 8 715 605, 2017.
- [33] Rasheed, I., Hu, F. i Zhang, L., „Deep reinforcement learning approach for autonomous vehicle systems for maintaining security and safety using LSTM-GAN”, *Vehicular Communications*, t. 26, s. 100 266, 2020.
- [34] Rastogi, M., Vijarana, M., Goel, N., Agrawal, A., Biamba, C. N. i Iwendi, C., „Conv1d-LSTM: Autonomous breast cancer detection using a one-dimensional convolutional neural network with long short-term memory”, *IEEE access*, t. 12, s. 187 722–187 740, 2024.
- [35] Shannon, C. E., „A mathematical theory of communication”, *The Bell system technical journal*, t. 27, nr. 3, s. 379–423, 1948.
- [36] Shenker, S. H. i Stanford, D., „Black holes and the butterfly effect”, *Journal of High Energy Physics*, t. 2014, nr. 3, s. 1–25, 2014.
- [37] Song, M., Zhu, Q., Peng, J. i Gonzalez, E. D. S., „Improving the evaluation of cross efficiencies: A method based on Shannon entropy weight”, *Computers & Industrial Engineering*, t. 112, s. 99–106, 2017.
- [38] Sparrow, C., *The Lorenz equations: bifurcations, chaos, and strange attractors*. Springer Science & Business Media, 2012.
- [39] Vernon, J. L., „Understanding the butterfly effect”, *American Scientist*, t. 105, nr. 3, s. 130, 2017.
- [40] Viana, M., *Lectures on Lyapunov exponents*. Cambridge University Press, 2014, t. 145.
- [41] Wędrawska, E., „Wykorzystanie entropii Shannona i jej uogólnień do badania rozkładu prawdopodobieństwa zmiennej losowej dyskretnej”, *Przegląd Statystyczny. Statistical Review*, t. 57, nr. 4, s. 39–53, 2010.
- [42] Wibig, J., „O współczynniku korelacji liniowej raz jeszcze”, W: K. Jarzyna (red.). *Zastosowanie metod statystycznych w geografii. Kielce: Instytut Geografii Uniwersytetu Jana Kochanowskiego w Kielcach*, s. 47–61, 2013.
- [43] Yazdanian, P. i Sharifian, S., „E2LG: a multiscale ensemble of LSTM/GAN deep learning architecture for multistep-ahead cloud workload prediction”, *The Journal of Supercomputing*, t. 77, nr. 10, s. 11 052–11 082, 2021.
- [44] Young, L.-S., „Mathematical theory of Lyapunov exponents”, *Journal of Physics A: Mathematical and Theoretical*, t. 46, nr. 25, s. 254 001, 2013.

- [45] Zhang, H., Goodfellow, I., Metaxas, D. i Odena, A., „Self-attention generative adversarial networks”, w *International conference on machine learning*, PMLR, 2019, s. 7354–7363.
- [46] Zhu, G., Zhao, H., Liu, H. i Sun, H., „A novel LSTM-GAN algorithm for time series anomaly detection”, w *2019 prognostics and system health management conference (PHM-Qingdao)*, IEEE, 2019, s. 1–6.

Spis rysunków

1	Schemat ogólnego sytemu komunikacji	12
2	Entropia Shannona	13
3	Wizualizacja dywergencji początkowych trajektorii	15
4	Wykres przedstawiający atraktor Lorenza	17
5	Schemat sieci GAN	18
6	Przebiegi czasowe napięć v_1 oraz v_2	21
7	Przebieg czasowy sygnału chaotycznego	31
8	Wykładnik Lapunowa	33
9	Entropia	34
10	Korelacje	36
11	Atraktor Lorenza	37